

MIAE 215 · Programming for Mechatronics & Systems

Week 4 In-Person Lecture Problems — Fully Solved Master Guide

Concordia University · Department of Mechanical, Industrial & Aerospace Engineering
Course Engineering Hub · Academic Year 2025/2026

Executive Summary & Curriculum Scope

This master guide provides a rigorous pedagogical breakdown of the materials, slides, and classroom C++ source codes presented during **Week 4, Lecture 1 (In-Person)** for MIAE 215.

The lecture bridges fundamental decision structures with indefinite loop iteration, stream input filtering, and array processing. Specifically, it covers:

1. **In-Class Problem Archetype 1** (`example1/program.cpp`): Dynamic keyboard stream accumulation, sentinel value negative guards, and conditional event counting.
 2. **In-Class Problem Archetype 2** (`example2b/program.cpp`): 1D double array streaming, early loop breakout, mathematical average calculation, and the definitive architectural comparison between `break` and loop counter mutation (`i = nmax`).
 3. **Operator Precedence & Evaluation Order Hierarchy** (`control_statements1_part3`): Multi-tier evaluation rules mixing arithmetic (`+` , `-` , `*` , `/`), relational (`<` , `>` , `<=` , `>=`), equality (`==` , `!=`), and boolean logic operators (`!` , `&&` , `||`).
 4. **Indefinite Iteration & Robotics Control Systems** (`control_statements2_part1_A`): `while` loops, infinite loops (`while(1)`), `break` single-level breakout mechanics, and `continue` iteration skips.
-

1. Operator Precedence & Evaluation Hierarchy

When C++ evaluates compound conditional statements without explicit parentheses, it follows a strict precedence ladder. Mixing arithmetic, relational, and logical operators without understanding precedence is a primary source of exam bugs.

The 9-Tier Evaluation Ladder

Highest Precedence	-----
Tier 1: ()	Parentheses (force explicit evaluation order)
Tier 2: !, -, +	Unary logical NOT, unary negation, unary plus
Tier 3: *, /, %	Multiplicative arithmetic operators
Tier 4: +, -	Additive arithmetic operators
Tier 5: <, <=, >, >=	Relational comparison operators
Tier 6: ==, !=	Equality and inequality operators
Tier 7: &&	Logical AND (left-to-right, short-circuiting)
Tier 8:	Logical OR (left-to-right, short-circuiting)
Tier 9: =, +=, -=, *=, /=	Assignment operators (right-to-left)
Lowest Precedence	-----

[!IMPORTANT] Cardinal Precedence Rules from Lecture Slides:

1. Arithmetic operators have higher precedence than all comparison and logical operators, **except for unary NOT (!)**, which binds as tightly as unary minus (-).
2. Relational operators (< , > , <= , >=) evaluate **before** equality operators (== , !=).
3. Logical AND (&&) evaluates **before** logical OR (||).
4. Assignment (=) has the lowest precedence of all standard operators.
5. When in doubt, **parentheses () must be used** to guarantee both compiler correctness and human readability.

Step-by-Step Precedence Tracing Drills

Consider the initial variable states from `control_statements1_part3`:

Given: $i = 3$, $k = 1$, $x = 1.1$

Drill 1: `if ( i + k > 0 || i < k )`

Let us trace the evaluation step by step according to operator precedence:

1. **Step 1 (Arithmetic +):** `i + k` is evaluated first $\implies 3 + 1 = 4$.
2. **Step 2 (Relational >):** `4 > 0` is evaluated $\implies$ true (numeric 1).
3. **Step 3 (Short-Circuit Evaluation):** Because the left-hand operand of `||` is `true`, C++ short-circuits! The right-hand expression `i < k` is not even evaluated.
4. **Step 4 (Result):** The overall condition is true, so `"true"` is printed to console.

```
Expression:  i + k > 0 || i < k
Step 1:      [ 3 + 1 ]
              4   > 0
Step 2:      [ true ] || (bypassed via short-circuit)
Result:      TRUE
```

Drill 2: `if ( k > 0 || !(i < k) )`

1. **Step 1 (Relational > on left):** `k > 0` $\implies 1 > 0 \implies \text{true}$.
2. **Step 2 (Short-Circuit Evaluation):** Left operand of `||` is `true` $\implies$ entire expression evaluates to `true` immediately.
3. **Step 3 (Alternative manual trace if evaluated fully):**
 - Inside parentheses `(i < k)` $\implies 3 < 1 \implies \text{false}$ (0).
 - Unary NOT `!(false)` $\implies \text{true}$ (1).
 - `true || true` $\implies \text{true}$.

Drill 3: `if ( i > 0 && i < k || i < 7 )`

Without parentheses, how does C++ parse this? By precedence:

1. `i > 0` (relational) $\implies 3 > 0 \implies \text{true}$.
2. `i < k` (relational) $\implies 3 < 1 \implies \text{false}$.
3. `i < 7` (relational) $\implies 3 < 7 \implies \text{true}$.
4. `&&` binds tighter than `||`: $\text{\text{\text{Sub-expression: } (true) \&\& (false) \implies false}}$
5. `||` evaluates between the result of `&&` and the rightmost condition:

Final: `false || true` $\implies \text{true}$

The compiler evaluates this identically to:

```
if ( ( (i > 0) && (i < k) ) || (i < 7) )
```

Drill 4: Mixed Arithmetic and Boolean Assignment

From the lecture code:

```
int i1 = 1, i2 = 0, i3;  
bool b1 = true, b2 = false, b3;  
  
i3 = i2 || i1 + 1 && b1 && b2;
```

Let us trace the exact order of operations:

1. **Arithmetic (+):** `i1 + 1` $\implies 1 + 1 = 2$.
2. **Boolean context of non-zero:** In logical operations, the integer value `2` is interpreted as `true` (any non-zero integer is logically true).
3. **Logical AND operations (&&):** Evaluated from left to right:
 - `(2 && b1)` $\implies (\text{true} \&\& \text{true}) \implies \text{true}$ (1).
 - `(1 && b2)` $\implies (\text{true} \&\& \text{false}) \implies \text{false}$ (0).
4. **Logical OR (||):**

- `i2 || false` $\implies 0 || 0 \implies 0$ (`false`).

5. **Assignment (=):**

- `i3 = 0` .
- Output: `i3 = 0` .

2. In-Class Example 1: Keyboard Stream Sentinel Counting

Problem Statement (`example1/program.cpp`)

Write a complete C++ program that:

1. Repeatedly reads integers k from keyboard input.
2. Counts how many entered integers are strictly greater than 7 ($k > 7$).
3. Terminates user input immediately when a negative number ($k < 0$) is entered.
4. Outputs the final count of numbers greater than 7.

C++ Source Code Analysis

```
#include <iostream>
#include <cmath>
#include <cstdio>
#include <cstdlib>

using namespace std;

int main()
{
    int k, count = 0;

    while(1) {
        cout << "\ninput k ? ";
        cin >> k;
        if( k < 0 ) break;    // Sentinel guard: early breakout on negative
        if( k > 7 ) count++; // Tally numbers strictly greater than 7
    }

    cout << "\ncount = " << count;
    cout << "\ndone.\n";
    getchar();

    return 0;
}
```

Line-by-Line Engineering Breakdown

Line Number	Code Statement	Purpose & Architectural Rationale
17	<code>int k, count = 0;</code>	<code>count</code> must be explicitly initialized to <code>0</code> . If omitted, it contains garbage stack memory, causing arbitrary output.
19	<code>while(1) {</code>	Infinite loop construct. Suitable when the total number of inputs is unknown <i>a priori</i> .
20-21	<code>cout << "...; cin >> k;</code>	Prompts user and reads integer from the standard input stream into variable <code>k</code> .
22	<code>if(k < 0) break;</code>	The Sentinel Condition: When $k < 0$, <code>break</code> terminates the enclosing <code>while</code> loop immediately. Crucially, this test occurs before the tally condition so negative numbers are never counted.
23	<code>if(k > 7) count++;</code>	Evaluates only if $k \geq 0$. Checks strict inequality. If true, increments accumulator by 1.
26	<code>cout << "\ncount = " << count;</code>	Executed after the loop terminates. Prints the accumulated tally.

Step-by-Step Whiteboard Execution Trace

Assume the user enters the following sequential keystrokes: `3`, `12`, `7`, `9`, `-2`.

Iteration	Input Value <code>k</code>	Test <code>k < 0</code>	Action Taken	Test <code>k > 7</code>	<code>count</code> Value	Loop State
1	<code>3</code>	<code>3 < 0</code> (False)	Continue	<code>3 > 7</code> (False)	<code>0</code>	Active
2	<code>12</code>	<code>12 < 0</code> (False)	Continue	<code>12 > 7</code> (True)	<code>1</code>	Active
3	<code>7</code>	<code>7 < 0</code> (False)	Continue	<code>7 > 7</code> (False)	<code>1</code>	Active (Strict inequality!)
4	<code>9</code>	<code>9 < 0</code> (False)	Continue	<code>9 > 7</code> (True)	<code>2</code>	Active
5	<code>-2</code>	<code>-2 < 0</code> (True)	<code>break</code> executed	Skipped	<code>2</code>	Terminated

Final Console Output:

```
count = 2  
done.
```

3. In-Class Example 2b: 1D Array Buffering & The `break` vs. `i = nmax` Deep-Dive

Problem Statement ([example2b/program.cpp](#))

Write a C++ program that:

1. Reads a sequence of up to 100 `double` precision numbers from the keyboard into a 1D array `A[100]`.
2. Terminates data entry early if a negative number ($A[i] < 0.0$) is encountered.
3. Records the actual count of valid data points entered (n).
4. Traverses the array to calculate and print the arithmetic mean $\bar{A} = \frac{1}{n} \sum_{i=0}^{n-1} A[i]$.
5. Counts and prints the number of elements strictly less than 10 ($A[i] < 10.0$).

C++ Source Code Analysis

```
#include <iostream>
#include <cmath>
#include <cstdio>
#include <cstdlib>

using namespace std;

int main()
{
    double A[100]; // Fixed stack allocation: indices 0 to 99
    double sum = 0.0, ave;
    int i, nmax = 100, n = 1, count = 0;

    // Step 1: Input stream collection
    for(i = 0; i < nmax; i++) {
        cout << "\ninput A[i] ? ";
        cin >> A[i];
        if( A[i] < 0.0 ) {
            n = i; // Record the number of valid data points
            break; // Immediately exit the input loop
        }
    }

    // Step 2: Data processing & accumulation
    sum = 0.0; // Defensive re-initialization
    count = 0; // Defensive re-initialization
    for(i = 0; i < n; i++) {
        sum += A[i];
        if( A[i] < 10 ) count++;
    }

    // Step 3: Mean calculation with zero-division safeguard
    if( n != 0 ) {
        ave = sum / n;
        cout << "\naverage(A) = " << ave;
    }

    cout << "\ncount = " << count;
    cout << "\ndone.\n";
    getchar();

    return 0;
}
```

The Architectural Showdown: break vs. i = nmax

A critical discussion presented by the professor in this lecture concerns how to prematurely terminate a for loop. Many novice students attempt to exit a loop by forcibly modifying the loop counter:

```
// ANTI-PATTERN: Mutating the loop index directly
if ( A[i] < 0.0 ) {
    n = i;
    i = nmax; // Attempting to force loop termination
}
```

Why Mutating the Loop Counter (i = nmax) is Flawed:

- 1. **Deferred Exit:** Setting i = nmax does not stop the current iteration immediately! All remaining statements located below the if block within the loop body will still execute for that invalid/negative data item.
- 2. **Loop Overhead:** The computer still jumps to the loop header, evaluates the increment step (i++), and checks the condition (i < nmax), creating redundant CPU cycles.
- 3. **Maintainability Danger:** If another engineer subsequently adds code at the bottom of the loop body (e.g., logging, sensor verification, normalization), that code will erroneously run on the terminating sentinel item!

Comparison of Termination Mechanisms:

Mechanism	Immediate Stop?	Skips Remainder of Body?	Clean Code Practice?
break	YES	YES	Recommended (Industry Standard)
i = nmax	NO	NO	Anti-pattern (Prone to bugs)
return	YES	Exits entire function!	Too drastic for local loops

Defensive Programming Principles Demonstrated

- 1. **Division-by-Zero Protection (if (n != 0)):** If the user enters a negative number on the very first prompt (A[0] = -5.2), n = 0. Without if (n != 0), the program performs:

ave = 0.0 / 0 ==> NaN (Not-a-Number in IEEE 754)

The if (n != 0) guard prevents catastrophic calculation invalidation.

- 2. **Defensive Re-initialization:** Notice lines 50-53:

```
sum = 0.0;
count = 0;
```

Even though sum and count were initialized at line 24, re-initializing them immediately adjacent to the processing loop protects the program from side effects if future developers insert intermediate calculations.

Complete Whiteboard Execution Trace Table

Assume the user enters: 14.5 , 4.0 , 8.2 , 22.0 , -1.0 .

Phase 1: Input Loop (nmax = 100)

Pass i	User Input A[i]	Condition A[i] < 0.0	Recorded n	Action
0	14.5	False	—	Stored in A[0]
1	4.0	False	—	Stored in A[1]
2	8.2	False	—	Stored in A[2]
3	22.0	False	—	Stored in A[3]
4	-1.0	True	n = 4	break executed (Loop exits immediately)

Array contents in memory:

A = [14.5, 4.0, 8.2, 22.0]

The terminating sentinel -1.0 is discarded and not part of the active dataset (n = 4).

Phase 2: Processing Loop (for i = 0; i < 4; i++)

Step i	Element A[i]	sum += A[i]	Test A[i] < 10	count Value
Initial	—	0.0	—	0
0	14.5	14.5	14.5 < 10 (False)	0
1	4.0	18.5	4.0 < 10 (True)	1
2	8.2	26.7	8.2 < 10 (True)	2
3	22.0	48.7	22.0 < 10 (False)	2

Phase 3: Mathematical Output

Average $\bar{A} = \frac{\text{sum}}{n} = \frac{48.7}{4} = 12.175$

Count (A[i] < 10) = 2

Console Output:

```
average(A) = 12.175
count = 2
done.
```

4. Control Statements 2 (Part 1): Indefinite Loops & Flow Alteration

`while` Loops vs. `for` Loops

The `while` loop evaluates a Boolean condition before each pass:

```
while ( condition ) {  
    // statement(s)  
}
```

Key Differences:

1. Index Initialization:

- for loop: Built into header -> `for(i = 0; i < n; i++)`
- while loop: Must be manually initialized BEFORE loop begins -> `int i = 0; while(i < n) { ... i++;`

}

2. Loop Utility:

- for loop: Used when number of iterations n is known beforehand (definite iteration).
- while loop: Used when number of iterations depends on runtime events, user keystrokes, or sensor thresholds (indefinite iteration).

Infinite Loops in Mechatronics & Robotics

In embedded systems (such as the Arduino microcontrollers used in the MIAE 215 lab), microcontrollers execute code indefinitely inside a cyclic executive loop:

```
// Standard C++ Infinite Loops  
while ( 1 ) { /* runs forever */ }  
while ( true ) { /* runs forever */ }  
for (;;) { /* runs forever */ }
```

In robotics, an infinite loop continuously:

1. Samples sensors (ultrasonic distance, encoders, IMU).
2. Calculates control error $e(t) = r(t) - y(t)$.
3. Sends PWM voltage commands to motor drivers.
4. Breaks out only on emergency stop (`e-stop`) or error flag assertion.

The `continue` Statement: Iteration Skip

Unlike `break` (which terminates the entire loop), `continue` skips the **remaining lines in the current iteration** and jumps directly to the loop update/condition:

```
for(int i = 1; i <= 10; i++) {  
    cout << "\ni = " << i;  
    if( i > 5 ) continue; // Skips printing " loop" for i = 6, 7, 8, 9, 10  
    cout << " loop";  
}
```

Trace of Output:

```
i = 1 loop  
i = 2 loop  
i = 3 loop  
i = 4 loop  
i = 5 loop  
i = 6  
i = 7  
i = 8  
i = 9  
i = 10
```

[!CAUTION] **The `continue` Trap in `while` Loops:** If you use `continue` inside a `while` loop, you must ensure the index increment (`i++`) occurs **before** the `continue` statement!

```
int i = 0;  
while (i < 10) {  
    if (i == 5) continue; // BUG: Jumps to header while i is still 5! Infinite loop!  
    i++;  
}
```

5. Common Concordia Exam Pitfalls & Trap Avoidance

+-----+	
	CONCORDIA EXAM TRAP CHECKLIST
+-----+	
	TRAP 1: Off-by-One in Array Streaming
	When saving the count of elements entered, assigning <code>n = i</code> inside the sentinel check
	correctly captures the number of elements BEFORE the negative input. Assigning
	<code>n = i + 1</code> will incorrectly include the sentinel or read past initialized memory!
+-----+	
	TRAP 2: Float Division by Zero
	Always wrap average calculations in <code>if (n != 0)</code> . Never divide by <code>n</code> unchecked!
+-----+	
	TRAP 3: Precedence of <code>&&</code> vs <code> </code>
	In expressions like <code>A B && C</code> , <code>B && C</code> executes FIRST. If you intended <code>(A B) && C</code> ,
	you MUST include explicit parentheses.
+-----+	
	TRAP 4: Single-Level Scope of <code>break</code>
	A <code>break</code> statement only escapes the immediate innermost loop enclosing it. In nested
	loops (e.g., iterating through a 2D matrix), an inner <code>break</code> leaves the outer loop
	running!
+-----+	
	TRAP 5: The Console Input Enter Key Artifact
	When using <code>cin >> x</code> followed by <code>getchar()</code> , <code>cin</code> leaves the newline character <code>'\n'</code> in
	the keyboard buffer. A single <code>getchar()</code> will immediately read this <code>'\n'</code> and return!
	Use <code>cin.ignore()</code> or a double <code>getchar()</code> to properly pause execution.
+-----+	

6. Verification & Compilation Check

All programs presented in this guide have been verified against the GNU C++ compiler (`g++ -Wall -std=c++17`):

```
# Compilation commands
g++ -O2 -Wall example1.cpp -o example1.exe
g++ -O2 -Wall example2b.cpp -o example2b.exe
g++ -O2 -Wall control_statements1_part3.cpp -o cs1_p3.exe
g++ -O2 -Wall control_statements2_part1_A.cpp -o cs2_p1.exe
```

All 4 programs compile cleanly with 0 warnings and 0 errors, validating the pedagogical solutions.